Lab Course Machine Learning

Exercise Sheet 12

Prof. Dr. Dr. Lars Schmidt-Thieme
Jung Min Choi

HiWi: Harish Malik
Submission deadline : January 25, 2025

1 Attention Layers (10 points)

1. **[1 Point]** Download the Iris dataset using `sklearn.datasets` with `load_iris`. Split the dataset into training and testing sets with an 80:20 ratio, and apply scaling to the dataset.
2. **[6 Points]** Implement a self-attention layer from scratch using PyTorch. The implementation should return both the outputs and the attention weights. Additionally, construct a simple classification model consisting of one self-attention layer followed by a fully connected layer. The model must return predictions and the corresponding attention weights.
3. **[3 Points]** Train the model and report the test accuracy. Furthermore, visualize the attention weights as a heatmap.

2 Transformer Text Generator (10 points)

1. **[1 Point]** Download the Harry Potter text data (`Harry_Potter_all_books_preprocessed.txt`) from Kaggle. Use only the first 50,000 lines (or tokens, if you prefer) for this assignment (You can use more data if you want.). Preprocess the text by converting it to lowercase and splitting it into tokens as necessary.
2. **[6 Points]** Implement
   a) A **Language Modeling Dataset** that divides the tokenized text into fixed-size chunks (e.g., sequences of length 50). Each sequence should predict the next token in a left-to-right manner (i.e., the label for position i is the token at position $i+1$).
   b) A **Transformer-Based Text Generator** from scratch using PyTorch, including:
      - An embedding layer for input tokens
      - Positional encoding
      - Multi-head self-attention (with a causal mask to prevent attending to future tokens)
      - Feed-forward layers
      - Residual connections and layer normalization
3. **[3 Points]** Train your model on this language modeling task using your preprocessed text. After training, demonstrate:
   - How to generate text given an initial prompt. For example, prompt the model with a starting word (e.g., *"from"*) and let it generate the following tokens until a specified length is reached. Implement a function `generate_text(prompt, max_length)` to accomplish this.
   - Show at least one illustrative example of the model's performance by calling `generate_text()` on a test prompt and printing the generated text.